

זרימת קניית מזון והממשק מול המשתמש

דוח מעמיק על תהליכי הבוט בטלגרם — מילוי עגלה, בחירת מוצרים, חוסרים ומבצעים, ודינמיקת ההודעות

מטרת המסמך: חומר גלם לתחקור עם מודל AI חיצוני. הדוח מתאר את התהליך כפי שהוא בנוי בקוד היום — לא כוונות, לא תוכניות — כדי לאפשר ביקורת עצמאית על העיצוב, נקודות התורפה, וההזדמנויות לשיפור.

נכתב: 2026-09-11

מקור: קריאה ישירה בקוד הפרויקט (grocery-automation) — כל פרט מצוין עם הפניה לקובץ

רשתות פעילות כיום: שופרסל, טיב טעם (ENABLED_STORES)

סטטוס: אוטומציה עד מילוי עגלה בלבד. תשלום/checkout נשאר תמיד ביד אדם — זהו כלל בלתי ניתן למשא ומתן בפרויקט.

תוכן עניינים

- 1. התהליך הידני לפני הבוט
- 2. עקרונות יסוד ומגבלת הבטיחות
- 3. איך המשתמש פוגש את הבוט — כל הפקודות
- 4. הבנת שפה חופשית (NLU) — מה קורה כשלא כותבים פקודה
- 5. הודעות יזומות — מתי הבוט פונה מיוזמתו
- 6. זרימת מילוי עגלה — שלב אחר שלב
- 7. בחירת מוצר: זיכרון, חיפוש, אי-ודאות
- 8. אי-ודאות מול המשתמש — איך נשאלת שאלה
- 9. מבצעים שנכנסים לעגלה לבד
- 10. חוסרים ותחליפים — תצוגת הסל
- 11. התחברות לאתרים וניהול session
- 12. כשלים ואיך הם מטופלים
- 13. זיכרון ולמידה מהיסטוריה
- 14. מה נמדד בפועל, ומה לא
- 15. נקודות פתוחות לתחקור

1. התהליך הידני לפני הבוט

מתועד ב- GOALS.md כבסיס לתכנון האוטומציה, לפני שנכתבה שורת קוד אחת. מובא כמעט מילה במילה, כי זה הבסיס להשוואה — מה בעצם נפתר ומה לא.

איך זה עבד

- קיימת רשימת מוצרים קבועה ("הבסיס") שנקנית כמעט תמיד.
- בכל מחזור קנייה: עוברים על הרשימה ומוסיפים רק את מה שבאמת חסר הפעם — רוב הפריטים כבר במלאי בבית ולא נדרשים בפועל.
- בקשות אד-הוק (בעיקר מלירן): או נזכרות בעל פה, או שמישהו נכנס לאפליקציה של הרשת ומכניס את הפריט לעגלה מיד — כדי לא לשכוח.
- אין מעקב מלאי אמיתי כלל. הכל מבוסס זיכרון: מה יש בבית, מה נגמר, מה כבר הוזמן.

שלוש נקודות הכאב שהוגדרו במפורש

1. אין בקרת מלאי — תלות מוחלטת בזיכרון. פריטים חסרים שאינם ברשימת הבסיס עלולים פשוט לא להיזכר, ולהתגלות רק כשחסרים בפועל.
2. קיבעון מותג/מוצר — תמיד נקנה אותו סוג מוצר ספציפי (למשל טונה מסוימת), כך שמבצעים או מותגים חלופיים משתלמים יותר מתפספסים באופן שיטתי.
3. בקשות אד-הוק לא ממוסדות — תלויות בזיכרון של מי ששמע את הבקשה, או בפעולה ידנית מיידית באפליקציה. אין ערוץ מרוכז אחד שאליו כל מי שבבית יכול לפנות.

מה עוד לא היה מוחלט כשהתכנון התחיל

הטריגר להתחלת מחזור הזמנה עצמו לא היה מוסכם: זמנן קבוע (שבועי/דו-שבועי), "לפי מה שבאמת נגמר" (דורש מעקב מלאי שלא היה קיים), או הפעלה ידנית לגמרי. ההחלטה בפועל, שהתגבשה מאוחר יותר, מתוארת בסעיף 5.

מה השתנה בפועל, ומה נשאר פתוח

מצב היום	כאב מקורי
עדיין אין מעקב מלאי אמיתי. הפתרון שנבחר הוא עקיף: "רשימת בסיס" רחבה ("באמת הכל") שממולאת כמעט במלואה בכל מחזור, והמחיקה — לא ההוספה — היא הפעולה הידנית. ר' סעיף 6.	אין בקרת מלאי
נפתר חלקית: זיכרון בחירה קבועה למוצר (סעיף 7) עדיין קיים במכוון — אבל מבצעים על מוצרים חדשים/לא-מוכרים נוספים לעגלה באופן יזום (סעיף 9), בדיוק כדי לשבור את הקיבעון.	קיבעון מותג/מוצר
נפתר: ערוץ אחד — הודעת טקסט חופשית לבוט — קולט גם בקשות מיידיות ("תוסיף X לעגלה") וגם תוספות לרשימה הפנימית ("תוסיף X"). ר' סעיף 4.	בקשות אד-הוק לא ממוסדות

לגבי הכרעת התדירות: תועד כי הרף לתזכורת ("עברו X ימים") ירד מ-6 ל-5 ימים ב-2026-09-07, אחרי שהזמנה קודמת (01/09) בוצעה ביוזמת המשתמש לפני שהתזכורת בכלל הגיעה — סימן שהרף המקורי היה גבוה מדי ביחס לקצב הקנייה בפועל של המשפחה.

2. עקרונות יסוד ומגבלת הבטיחות

הכלל היחיד שאינו ניתן לשינוי: האוטומציה מגיעה עד מילוי עגלה בלבד. שום נתיב קוד לא מגיע לשלב תשלום או אישור הזמנה סופי. זה מתועד כדרישת-על ב- CLAUDE.md וגם ברמת קוד: `adapters/selfpoint.py` מתואר כ"קריאת מחיר בלבד — במכוון בלי חשבון, עגלה או checkout", ו- `adapters/tivtaam_api.py` מסיר במפורש כל טוקן תשלום מתוך payload-ים ("strip_payment") ומצהיר שאין בו שום מתודה שמגיעה לשלב תשלום.

גבולות נוספים שנקבעו לגבי חשבון החנות המחובר (זהים לכל רשת, ללא יוצא מן הכלל):

- **מותר:** חיפוש קטלוג, קריאת מחירים ומבצעים, קריאת היסטוריית הזמנות ורשימות שמורות, הוספה/הסרה בעגלה.
- **אסור בהחלט:** הזנה/עריכה/קריאה של פרטי כרטיס אשראי; שינוי פרטי זהות או הגדרות חשבון (אימייל, טלפון, כתובת, סיסמה, מועדון, העדפות שיווק); שיטוט בתפריטי חשבון מעבר למילוי ההזמנה; מימוש קופונים או הטבות; ביטול/שינוי הזמנות קיימות.

עקרון עיצוב מרכזי נוסף: "נקודת החלטה ממוקדת, רק על אי-ודאות אמיתית" — המשתמש נשאל שאלה רק כשיש עמימות אמיתית (למשל שני מוצרים תואמים במידה שווה), ולא בכל צעד. זה מה שמאפשר למחזור שלם לרוץ כמעט ללא התערבות (ר' סעיף 7).

3. איך המשתמש פוגש את הבוט — כל הפקודות

מקור: CommandHandler ב-15 — grocery_bot/telegram_bot.py פקודות מוגדרות בפועל.

פקודה	מה היא עושה
/start	אתחול השיחה עם הבוט
/start_order	מריץ מחזור מילוי עגלה מלא, על כל מה שממתין ברשימה ובבקשות האד-הוק
/list	מציג את הרשימה הקבועה (הבסיס) הפעילה
/list_full	יצירת/הצגת רשימה מלאה (טקסט להעתקה)
/price	מחיר מדף בודד ברשת אחת (שופרסל)
/deals	מבצעים על מה שברשימה הקבועה
/alldeals	רשימת מבצעים מורחבת
/chaindeals	מבצעים חריגים ברשתות אחרות שאינן העגלה הפעילה
/basket	תצוגת הסל המלא בכל רשת, עם חוסרים ותחליפים מסומנים (ר' סעיף 10)
/lastdeals	אילו מבצעים נוספו לבד במחזור האחרון
/done	"סיימתי לקנות" — רושם שהקנייה בוצעה, ומפעיל מילוי-מחדש אוטומטי של העגלה (ר' סעיף 6)
/cheaper	מוצרים זהים/דומים במחיר נמוך יותר ברשת אחרת
/stockup	הצעות "לאגור" — מוצרים ברי-אגירה במבצע עמוק
/refresh_prices	רענון ידני של פיד המחירים והמבצעים מכל הרשתות
/digest	סיכום מצב יזום — אותו תוכן שנשלח אוטומטית (ר' סעיף 5)

כפתורים (לחיצה, לא הקלדה): CallbackQueryHandler — כפתור אישור כרטיס הטבה, כפתור מבצעי-רשתות, כפתור מבחר הצעה (proposal), כפתור מתכון, כפתור דיווח בזבז, ופתרון אי-ודאות מוצר (ר' סעיף 8).

4. הבנת שפה חופשית (NLU) — מה קורה כשלא כותבים פקודה

מקור: `grocery_bot/loop.py`, `grocery_bot/nlu.py`. הרוב המכריע של ההודעות אינן פקודות עם "/" — הן משפט חופשי בעברית.

המסווג הראשי

כל הודעת טקסט חופשית (שאינה פקודה) עוברת דרך `parse_message`, שקורא למודל שפה (`claude -p`), על המנוי הקיים של המשתמש — לא מפתח API נפרד) ומחזיר אחת מ-13 כוונות אפשריות:

```
add_item · remove_item · price_query · deals · show_list · recipe · meal_plan · start_order ·  
add_to_cart · report_waste · shopped · smalltalk · unclear
```

זמן תגובה טיפוסי: **9-7 שניות** (נמדד). קיים מפריד חד-משמעי בין "רשימה" ("הרשימה הפנימית של הבוט, שממתינה למחזור הבא") לבין "עגלה/סל" ("הסל האמיתי באתר") — הבחנה שמונעת בקשות כפולות בטעות.

מסלול גיבוי — כשהמודל לא זמין

אם קריאת המודל נכשלת (`timeout`, ה-CLI לא נמצא וכו') — מסווג חוקים פשוט תופס, שמזהה קידומות כמו "תוסיף"/"תוריד"/"כמה עולה", ומעדיף להחזיר `unclear` על ניחוש. זה תיקון לבאג עבר: ללא נתיב חילוץ ל-CLI, כל הודעה נפלה בשקט למסלול הבסיס, וגם "תכנן לי תפריט שבועי" נכנס לרשימת הקניות כפריט בשם "תכנן לי תפריט שבועי".

"הלולאה" — מחשבה שנייה על הודעה עמומה (נבנה 2026-09-09)

כאשר המסווג הראשי מחזיר `unclear`, מתבצע מעבר שני שמקבל הקשר מוזן מראש (רשימת הבסיס והבקשות הממתינות, נקראות ישירות ממסד הנתונים) ומנסה לפתור את העמימות — למשל "הלבנה נגמרה" מול רשימה שמכילה "לבנה" ספציפית.

מחסום בטיחות מובנה בקוד, לא בהנחיית המודל: תוצאה שהמעבר השני מציע ל-`add_to_cart`, `start_order` או `shopped` — נדחית תמיד, ומוחזרת כ-`unclear` עם שאלה חוזרת. הסיבה: הודעות שמגיעות לשלב הזה הן בדיוק כאלה שאיש לא הבין; להפוך הודעה בלתי-ניתנת-לפענוח לפעולת עגלה אמיתית היא הכיוון הלא נכון לפתור בו ספק. (`shopped` נכלל כי הוא מפעיל מילוי-מחדש אוטומטי של שתי העגלות האמיתיות).

תקרת זמן: 30 שניות (הורדה מ-90 אחרי שנמדד מקרה שלקח במוצע 62.7 שניות על פני שלוש ריצות — יותר מדי המתנה עבור הודעה שהתוצאה הכי טובה שלה היא שאלת הבהרה).

5. הודעות יזומות – מתי הבוט פונה מיוזמתו

מקור: `grocery_bot/telegram_bot.py` (`job_queue`), `grocery_bot/nudge.py` , `grocery_bot/learn.py`

מתי	שם	מטרה
כל 2 דקות	<code>drain_deferred_cycle</code>	ממשיך מחזור מילוי שנדחה (למשל בגלל שהיציאה הישראלית הייתה למטה)
יומי, 18:40 שעון ישראל	<code>cadence_check</code>	בודק אם הגיע הזמן לפתוח שיחה יזומה, לפי הקצב האמיתי של המשפחה – לא יום קבוע בשבוע
לילי, 03:40	<code>nightly_learn</code>	לומד מהיסטוריית ההזמנות האמיתית של הרשת (כולל הזמנות שבוצעו ידנית לגמרי באפליקציה)

שעת 18:40 נבחרה במפורש

"מספיק מאוחר שתוכניות היום כבר ידועות, מספיק מוקדם כדי להזמין לפני שחלונות המשלוח מתמלאים" – מתועד כהחלטה מודעת, לא ברירת מחדל.

תוכן ה"תזכורת" (nudge) כשעברו 5 ימים מהזמנה אחרונה

הנוסח "העגלה כבר מוכנה" נבחר ב-2026-09-07 מתוך שלוש טיוטות. ההודעה נשלחת דרך הבוט המשפחתי המשותף (לא בוט הקניות עצמו) – כדי שלא לדרוש מבני הבית לפנות לעוד עוזר.

עברו 5 ימים – העגלה כבר מוכנה 🛒

מילאתי 34 פריטים בשופרסל ו-19 פריטים בטיב טעם לפי מה שאתם קונים בדרך כלל.

כנראה נגמר במזווה:

- חלב 3%
- ביצים

[תזכורת טעינת כרטיס ההטבות, אם רלוונטי החודש]

מבצעים חריגים שכדאי לדעת עליהם:

[רשימת מבצעים, אם יש]
[עוד N מבצעים](קישור טלגרם)

מה שנשאר: לעבור, להוריד מה שלא צריך, לשלם.

🛒 פתיחת הסל בשופרסל]

🛒 פתיחת הסל בטיב טעם]

אם עדיין לא מולאה עגלה, ההודעה שונה לחלוטין – פותחת בשאלה פתוחה ("מה להוסיף לקנייה הבאה?") במקום בדיווח מצב.

הכרעת "האם מגיע השיחה עכשיו" (cadence_check)

מבוססת על הפער האופייני האמיתי בין הזמנות של המשפחה (`typical_gap_days`), לא על מספר קבוע. לא נשלחת פעמיים לאותו מחזור הזמנה – לאחר שנשלחה, מחכה להזמנה חדשה או לשלושה ימים נוספים.

תזכורת כרטיס הטבות

נרכבת על אותה הודעה בדיוק, כי היא חייבת להגיע לפני הקנייה – "וזו ההודעה שכבר מגיעה אז" (הנמקה מתועדת בקוד).

6. זרימת מילוי עגלה — שלב אחר שלב

מקור: `grocery_bot/orchestrator.py` (`run_order_cycle`), `grocery_bot/standingcart.py`

הטריגר: "עגלה תמיד מלאה"

ההחלטה הסופית (2026-09-09, בעקבות דיון עם המשתמש): העגלה מתמלאת מחדש אוטומטית **אחרי כל דיווח קנייה** ("סיימתי לקנות" / `/done`), ולא לפי זמנון קבוע. במהלך השבוע ניתן להוסיף/להסיר ידנית; הפעולה הידנית העיקרית שנותרה למשתמש היא **מחיקה** לפני התשלום, לא הוספה.

1. יף המקורות. לכל רשת מופעלת בנפרד: רשימת הבסיס הפעילה ("הבסיס"/"באמת הכל") + בקשות אד-הוק ממתינות + מבצעים שנבחרו להוספה יזומה (סעיף 9). אם לרשת מסוימת אין עדיין היסטוריית קנייה משלה (למשל טיב טעם כשהתחילו), נופלים חזרה לרשימת הבסיס הכללית של המשפחה — כדי שגם רשת חדשה תקבל עגלה מלאה ולא ריקה.

2. ש session. לפני שמתחילים לגעת בפריטים, מוודאים שההתחברות לרשת עדיין תקפה ומחדשים אם פג תוקף (ר' סעיף 11). ללא זה, כל פריט היה נכשל עם הודעת שגיאה שנראית כמו סלקטור שבור, ולא כמו מה שהיא באמת.

3. ר מוצרים בבת אחת (bulk match). כל המונחים החדשים (שאינן עבורם עדיין בחירה זכורה) נשלחים בקריאה אחת למנגנון ההתאמה של הרשת, במקום טעינת דף נפרדת לכל פריט. שיפור מדוד: 89 פריטים שלקחו כ-15 דקות בחיפוש בזה-אחר-זה ירדו ל-כ-40 שניות.

4. פה בפועל, פריט-פריט (ר' פירוט מלא בסעיף 7: סדר ההחלטה — זיכרון → התאמה מרוכזת → חיפוש חי → אי-ודאות).

5. ון התקדמות חי. קריאה חוזרת (`on_progress`) אחרי כל פריט מאפשרת לעדכן הודעה אחת בטלגרם עם ההתקדמות, כדי שמחזור של כמה דקות לא יראה כמו שיחה תקועה.

6. ת מעגל. בקשת אד-הוק נמחקת מהרשימה רק אם משהו הצליח בפועל איתה (נוספה או נענתה שאלת אי-ודאות) — כישלון זמני (תקלת רשת, שינוי במבנה האתר) לא ימחק בשקט בקשה שאיש לא יזכור לשלוח שוב.

7. מ כישלונות (חדש, 2026-09-10). כל פריט שלא הגיע לעגלה נשמר בטבלה ייעודית עם חותמת זמן ורשת. אם אותו פריט נכשל ב-3 מחזורים נפרדים ומעלה, שורה מיוחדת ("נכשלים שוב ושוב") מופיעה בדוח — כדי להבחין בין החמצה חד-פעמית (בדרך כלל תקלת חיפוש של הרשת) לבין דפוס אמיתי (מוצר שירד מהמדף, או מונח חיפוש שגוי).

8. סיכום למשתמש (ר' תוכן לדוגמה בהמשך).

נתיב נפרד: הוספה מיידית לעגלה ("תוסיף X לעגלה")

מובחן במכוון מהמחזור המלא: מריץ רק את הפריטים שהוזכרו בהודעה הספציפית, כדי שלא "יישפכו" עוד עשרות מוצרים שהמשתמש לא ביקש באותו רגע.

דוח סיכום — דוגמה לתוכן

✓ נוספו (31)

🔑 נוספו בגלל מבצע חריג (4) — מחקו מה שלא צריך:

• יוגורט יווני 1 ק"ג — 2 ב-20 ₪, במקום 15.90 ליחידה

? דורש בחירה (2): קוטג', חלב סויה

⚠ לא נמצא (1): פטרוזיליה

נשאר ברשימה — אנסה שוב בפעם הבאה.

🔄 נכשלים שוב ושוב (1) — כנראה ירדו מהמדף או שהמונח לא מדויק:

• במבה אישית (shufersal) — 5 מחזורים, מאז 2026-09-05

אפשר לתקן את המונח, או להוריד מהרשימה.

7. בחירת מוצר: זיכרון, חיפוש, אי-ודאות

מקור: `_add_one` ב- `grocery_bot/orchestrator.py`.

לכל מונח (למשל "חלב 3%"), סדר ההחלטה הוא הבא, ונעצר בשלב הראשון שמצליח:

1. זכרה (`product memory`). אם המשפחה כבר ענתה על שאלת אי-ודאות בעבר עבור מונח זהה באותה רשת, המוצר המדויק שנבחר אז מתווסף ישירות — בלי חיפוש, בלי שאלה חוזרת. זהו הבסיס שהופך את הבוט לשישי: חיפוש רגיל של מונח יומיומי מחזיר כ-20 תוצאות, ובלי זיכרון כל פריט היה הופך לשאלה.

2. חכמה אם הבחירה הזכורה נעלמה. אם המוצר הזכור הורד מהמדף/שונה שם, לא מדווח כישלון מיידי — מתבצע חיפוש טרי, והזיכרון נשאר (לא נמחק על כישלון בודד; כישלון חד-פעמי בדרך כלל אומר שהחיפוש של הרשת לא מצא הפעם, לא שהמשפחה שינתה דעתה).

3. זמה מרוכזת (`prematched`, מהשלב 3 בסעיף 6). אם המונח כבר נפתר בקריאת ה-`bulk-match` הראשונית לקוד מוצר יחיד — הוספה ישירה. אם המוצר שסומן אזל מהמלאי, מדווח "לא נמצא" עם הפירוט "אזל מהמלאי" ולא מנסה בכוח.

4. ש חי באתר. אם עדיין אין תשובה — חיפוש רגיל בזמן אמת מול הרשת.

5. טה על התוצאה: תוצאה אחת חד-משמעית → נוספת ישירות. כמה מועמדים תואמים באותה מידה → נשמרת שאלת אי-ודאות (סעיף 8) ולא נבחר דבר אוטומטית. אין תוצאה כלל → "לא נמצא", ונשאר ברשימה למחזור הבא.

מדוע לא נבחר תמיד המוצר הראשון בתוצאות: ניסיון קודם שבו נבחרה "הכי זולה" באופן אוטומטי גרם למקרה של קניית תוצר "כמות פחותה נראית הכי זולה" — כלומר אריזה קטנה יותר ניצחה במחיר ליחידה בטעות. ההיגיון הזה תועד ותוקן; ההעדפה כיום, כשיש כמה תחליפים שווי-ערך, היא למוצרים בפיקוח מחיר (מוצרי יסוד בפיקוח ממשלתי) — לא למחיר המספרי הזול ביותר גרידא.

8. אי-ודאות מול המשתמש — איך נשאלת שאלה

מקור: `resolve_ambiguity`, `_format_choice` ב- `grocery_bot/telegram_bot.py`.

שאלת אי-ודאות מוצגת כרשימה ממוספרת עם אימוג'ים, וכפתורים תואמים מתחתיה — לא צריך להקליד תשובה:

קוטג' — איזה מהם?

1 קוטג' 5% שומן, 250 גרם — ₪ 6.90 💰

2 קוטג' 3% שומן, 250 גרם — ₪ 7.50

3 קוטג' תנובה 9%, 250 גרם — ₪ 8.20

[1] [2] [3] [✓] סיום

💰 מסמן את האפשרות הזולה ביותר מבין המועמדים, כשירות למשתמש — אך לא נבחרת אוטומטית.

עיצוב מכוון: בחירה מרובה, השאלה נשאלת פתוחה

לחיצה על מספר **מוסיפה מיד** את הווריאנט הזה לעגלה, והשאלה נשאלת פתוחה — כי המשפחה אכן קונה גם 5% וגם 3% באותה קנייה לפעמים. רק "✓ סיום" סוגר את השאלה. עיצוב זה בא לתקן חוויה קודמת שבה משתמש היה מסמן ואז צריך לאשר — והמסך לא הראה שום דבר קורה מיד עם הלחיצה.

הכפתור מאושר תוך פחות מ-15 שניות ממועד הלחיצה (מגבלת טלגרם על תוקף כפתור) — ולכן האישור מתבצע לפני כל פעולה איטית מול הרשת עצמה, כדי שהכפתור לעולם לא "יראה מת".

ניקוי שאלות שהתיישנו

לפני שליחת שאלה ממתינה, נבדק שוב אם המשפחה כבר "ענתה" עליה בדרך אחרת בינתיים (בחירה שנלמדה מייבוא היסטוריה, למשל) — כדי לא לשאול שוב שאלה שכבר נענתה בפועל.

9. מבצעים שנכנסים לעגלה לבד

מקור: `grocery_bot/radar.py`, `grocery_bot/dealfill.py`.

"עדיף שנצא פחות מאשר שנאכל עם 20% הנחה" — אך ההיגיון הזה מתייחס לקניות פנאי וטכנולוגיה דרך הטבות כרטיסי אשראי, **לא לסופר**. בסופר המשתמש מבקש במפורש לראות ולנצל מבצעים.

ההיגיון המרכזי: מבצע שמגיע רק כהודעת טקסט הוא מבצע שדורש פעולה שנייה כדי לממש — ונמדד בפועל שהפעולה השנייה הזו לא קורית. סף המתנה למתנה (₪599) הוחמץ פעמיים ברצף לפני שהתכונה נבנתה (בפער של ₪38.21 וב-₪77.81), כי נקנתה רק יחידה בודדת ממוצר שהיה עליו "2 ב-1", בלי שמישהו שם לב. לכן: מבצעים משמעותיים נכנסים **ישירות לעגלה**, ולא רק לדוח — כי מחיקת שורה מעגלה לוקחת שנייה, וזיהוי מבצע ידנית בדרך כלל פשוט לא קורה.

שני סוגי מבצעים, שתי מדיניות שונות

מוצרים "חדשים" (לא נקנו מעולם)	מוצרים שהמשפחה כבר קונה	
40%	25%	סף הנחה מינימלי
₪30	—	תקרת מחיר
80% (חשד להטעיה מעל זה)	—	תקרת הנחה
אותו עיקרון: רק ברי-אגירה, אף פעם לא מוצרים מתכלים	טבלת קניות אמיתית של המשפחה בלבד	מקור המועמדים

שתי הגנות שמונעות בזבוז (המדד החשוב יותר בעיני הפרויקט מהחמצת חיסכון)

- **רק ברי-אגירה כברירת מחדל**. מרכז כביסה ב-40% הנחה יושב במזווה; בננות בחצי מחיר נרקבות. מסווג לפי מחלקת מוצר; מתכלים מדווחים בלבד, לא נכנסים לעגלה לבד.
- **רק דברים שהמשפחה קונה בפועל, בשוטף**. מאגר המועמדים הוא טבלת ההיסטוריה האמיתית, לא הקטלוג הכללי — הנחה עמוקה על מוצר שמעולם לא נקנה היא לא חיסכון, היא הרגל חדש שאף אחד לא ביקש.

שיוך מבצע לרשת הנכונה בלבד: מבצע שהתפרסם בשופרסל נוסף רק לעגלת שופרסל — הכנסתו לעגלת טיב טעם הייתה גורמת לרכישה במחיר המלא שם; ההנחה אינה ניתנת להעברה. שתי הרשתות נבדלות גם בשיטת ההתאמה: לשופרסל אין ברקוד ציבורי אז ההתאמה היא לפי שם; לטיב טעם יש ברקוד גם במחירים וגם במבצעים דרך הפיד הציבורי, כך שההתאמה שם ודאית ולא יכולה לזהות מוצר לא נכון.

שקיפות: תיוג בדיעבד

כל שורה בעגלה שנוספה בגלל מבצע (ולא כי המשפחה ביקשה אותה) מסומנת ככזו בדוח ובפקודת `/lastdeals`, כדי שהמשתמש יוכל להבדיל בקלות בין מה שהוא ביקש למה שהמערכת הוסיפה מיוזמתה — נבנה אחרי שהמשתמש שאל בפירוש "איך אני יודע מה נוסף בגלל מבצע?" ולא הייתה תשובה זמינה.

10. חוסרים ותחליפים — תצוגת הסל

מקור: `grocery_bot/basketview.py`, פקודת `/basket`.

מציג את כל רשימת הבסיס מתומחרת בכל רשת בו-זמנית, כדי לאפשר השוואה לפני שממלאים בפועל, עם שלוש רמות סימון לכל פריט:

סימן	משמעות
✓	נמצאה התאמה מדויקת — מוצר ששמו מתחיל בדיוק בשם המבוקש
↪	נמצא תחליף (לא התאמה מדויקת)
✗	אין שום מוצר מקובל באותה רשת

כלל קפדני: תחליפים מוצגים אבל **אף פעם לא נכנסים לחישוב המחיר הכולל** (`exact_subtotal`). הסיבה המתועדת: ריצה ראשונה של התכונה הראתה בדיוק את המלכודת ההפוכה — רשת שבה חלק גדול מהמוצרים לא במלאי נראתה "הכי זולה" רק כי פחות פריטים בכלל נכנסו לסכימה. המחיר הכולל המדווח משקף אך ורק את מה שבאמת נמצא, כדי לא להטעות בהשוואה בין רשתות.

לצד זה, `/lastdeals` ופקודות המבצעים מציגות פער בין "מחיר בסיס" (`baseline`) למחיר בפועל אחרי תחליפים/מבצעים — כך שהמשתמש יכול לראות גם היכן המערכת פשרה על תחליף זול יותר.

11. התחברות לאתרים וניהול session

מקור: `grocery_bot/adapters/shufersal.py`, `tivtaam.py`, `grocery_bot/exitnode.py`.

מוצא ישראלי (exit node) — תלות תשתית קריטית

שופרסל וטיב טעם חוסמים כתובות IP שאינן ישראליות (החזרת דף חסימה עם HTTP 200 — לא שגיאה גלויה, מה שמסביר תקלות שנראו במשך זמן כמו סלקטורים שבורים). הפתרון: יציאת Tailscale ישראלית דרך מכשיר בבית המשתמש (בפועל: האייפון האישי, כי קופסת ה-Android TV הייתה לא זמינה תקופה ארוכה). כל בקשה לאתרי הרשתות עוברת פיזית דרך חיבור הסלולר/ביתי של המשתמש. פיד המחירים הציבורי (חוק שקיפות המחירים) **אינו** עובר דרך היציאה הזו במכוון — כדי לא לצרוך את רוחב הפס הביתי סתם.

לפני כל מחזור, המערכת בודקת שהיציאה בריאה ומחליפה צומת אם צריך. תקלה כאן חסמה בעבר מחזורים שלמים עם timeouts שנראו כמו בעיה אחרת לגמרי.

התחברות (login)

שם משתמש+סיסמה בלבד — אומת מול חשבון אמיתי, בלי שלב אימות SMS/קוד באף אחת משתי הרשתות הפעילות. Session נשמר ומחודש אוטומטית; אם החידוש נכשל, המחזור מדווח שגיאה מפורשת ("Session expired") במקום להיכשל על כל פריט בנפרד באופן שנראה כמו סלקטורים שבורים.

12. כשלים ואיך הם מטופלים

הודעה שנעלמת בגלל תבנית טקסט (Markdown)

שני מקרים אמיתיים תועדו: כפתור מבצעים שהלחיצה עליו לא הראתה שום דבר (השליחה נכשלה בשקט), והרבה יותר חמור — סיכום הזמנה אמיתית שנעלם אחרי ששתי העגלות כבר התמלאו, יחד עם 39 שאלות אי-ודאות שלא נשלחו. תוקן בשתי שכבות: (א) מעבר הדרגתי לפורמט HTML שבו תווים כמו * ו- & לא שוברים את ההודעה; (ב) מנגנון גורף — כל שליחת הודעה בפרויקט (+40 נקודות קוד) עברה לעטיפה שמנסה לשלוח עם עיצוב, ואם טלגרם דוחה — שולחת מחדש כטקסט פשוט. "ההודעה תגיע פחות יפה" במקום "ההודעה אבדה".

כשל שקט מול כשל מדווח

עיקרון חוזר בקוד: **עדיף שגיאה גלויה על תוצאה שגויה שנראית תקינה**. דוגמאות: פריט שלא נמצא נשאר מסומן במפורש ("לא נמצא, ינוסה שוב"), קטלוג הטבות שלא ניתן לקריאה מציג "לא ידוע" ולעולם לא "0%" (כדי לא להיראות כמו "אין הנחה"), ותקרת ארנק כספי שפג תוקפה מוסרת אוטומטית מרשימת ההצעות במקום להמשיך ולהציג אחוז הנחה שכבר לא קיים.

13. זיכרון ולמידה מהיסטוריה

מקור: `nightly_learn`, `grocery_bot/learn.py`.

מדי לילה (03:40), הבוט קורא את היסטוריית ההזמנות האמיתית של הרשת — כולל הזמנות שבוצעו לגמרי ידנית באפליקציה, לא רק דרכו. כך המודל של "מה המשפחה קונה בדרך כלל" נשאר מעודכן בלי מאמץ נוסף, גם כשמישהו פשוט פותח את האפליקציה וקונה בעצמו. זה גם המקור ל"פער הימים האופייני" שקובע מתי לשלוח את השיחה היזומה (סעיף 5), ולרשימת "מה נגמר במזווה" בהודעת התזכורת (מבוסס משך חיי מדף משוער לכל מוצר).

14. מה נמדד בפועל, ומה לא

נמדד ומתועד

- זמן תגובה של סיווג כוונה: 7-9 שניות; מעבר-מחשבה-שני: 12-17 שניות בדרך כלל.
- ריצת 28 ניסוחים $\times$ 3 חזרות: 84/84 סיווגים נכונים, ללא חוסר יציבות (בהגדרות ברירת המחדל של המודל).
- שיפור מהירות התאמת מוצרים מרוכזת: כ-15 דקות $\rightarrow$ כ-40 שניות ל-89 פריטים.
- שני מקרי הפער בסף המתנה (₪38.21, ₪77.81) שהניעו את מנגנון המבצעים היזום.

לא נמדד / לא ידוע כיום

- אין עדיין מדגם מספיק של "נכשלים שוב ושוב" בפועל — המנגנון (סעיף 6) הופעל רק ב-2026-09-10; התובנה הראשונה תהיה זמינה אחרי כמה מחזורים.
- אין מדד כמותי לשביעות רצון/תדירות מחיקה מהעגלה בפועל — כלומר לא ידוע כמה מתוך המבצעים והפריטים הכלליים שנוספים לבד באמת נשארים בעגלה מול נמחקים.
- עלות תפעולית (זמן מודל, קריאות CLI) עדיין לא נמדדת בפרויקט זה במונחי לוג מובנה — הוחלט במפורש (-2026-09-09) לא לבנות זאת כרגע.

15. נקודות פתוחות לתחקור

אלה שאלות עיצוב שהמסמך הזה חושף אך אינו פותר — מיועדות לדין עם המודל החיצוני:

1. **עומס קוגניטיבי הפוך.** העיקרון "עגלה תמיד מלאה, המחיקה על המשתמש" פותר את בעיית ה"שכחתי להוסיף" אבל מעביר את העול לכיוון ההפוך — האם יש סיכון ל"עייפות מחיקה" (deletion fatigue) שגורמת לקניית דברים שלא רוצים בפועל, בדיוק ההיפך מהכוונה המקורית?
2. **זמן תגובה של 7-30 שניות לכל הודעה.** האם זה חסם אמיתי לשימוש שוטף באמצע יום עמוס, במיוחד כשמעבר-המחשבה-השני נדרש?
3. **מבצעים יזומים כמנגנון נגד קיבעון מותג — האם הוא מספיק?** הסף (40% הנחה, עד 30%, למוצרים חדשים) הוא ניחוש מבוסס-הגיון, לא נגזרת ממדידה בפועל של מה שהמשפחה שומרת מול מה שהיא מוחקת.
4. **תלות ביציאה ישראלית יחידה (טלפון אישי).** נקודת כשל יחידה שכל המחזור תלוי בה, ושאינה בשליטת הבוט עצמו.
5. **המעבר בין "רשימה" ל"עגלה" בשפה טבעית** — האם ההבחנה ברורה מספיק למשתמש בפועל, לאורך זמן, או שהיא דורשת זיכרון של מוסכמה?